

Message Flows

This section explains the end-to-end flow of messages in the chat system. We discuss:

- 1 on 1 chat flow
- Message synchronization across multiple devices
- Small group chat flow

1 on 1 Chat Flow

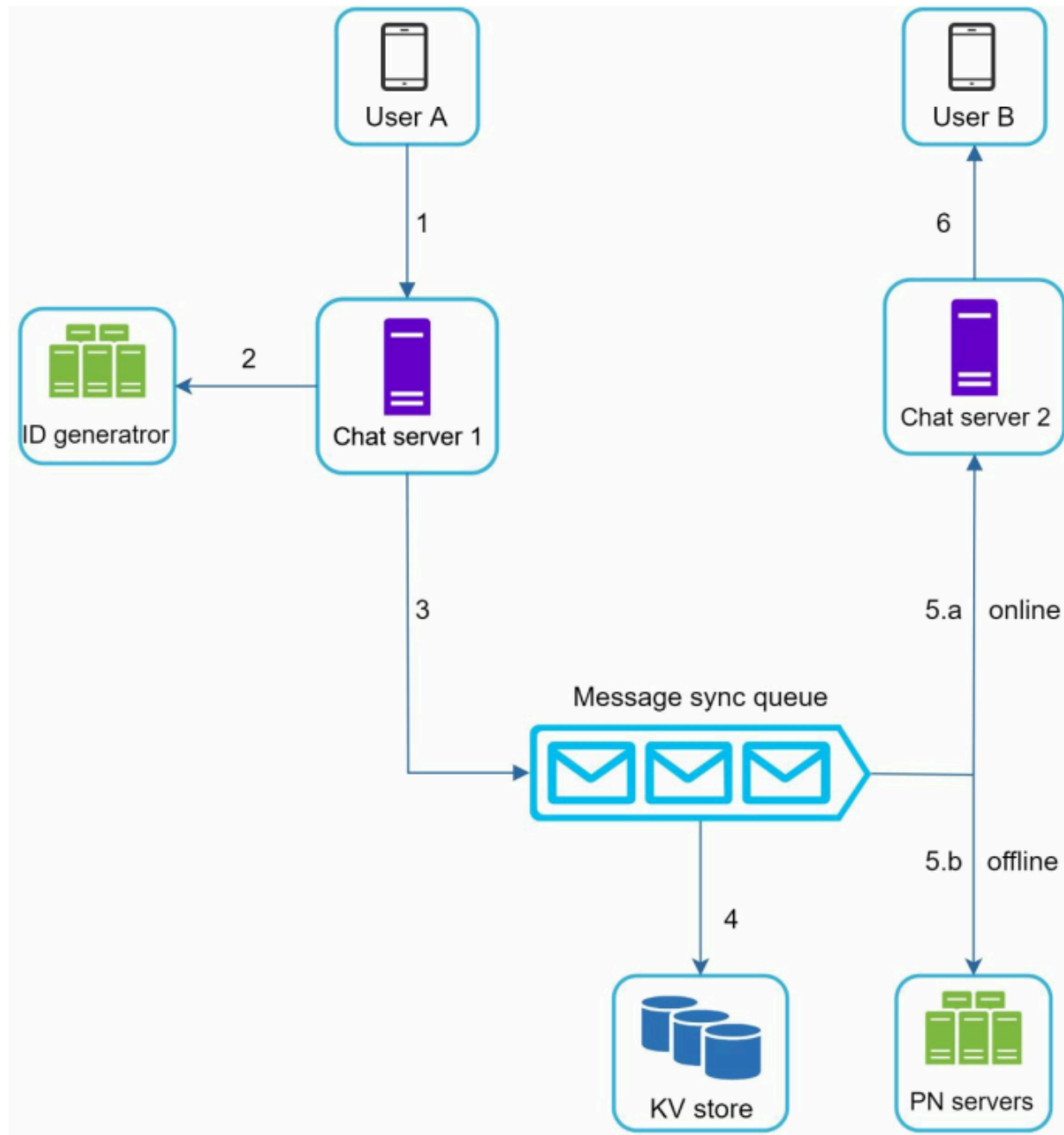


Figure explains the flow when **User A** sends a message to **User B**.

Workflow

Step 1: Send Message

User A sends a chat message to **Chat Server 1** through a persistent WebSocket connection.

Step 2: Generate Message ID

Chat Server 1 requests a unique **message_id** from the ID Generator.

Step 3: Add to Message Sync Queue

The message is pushed into the **message synchronization queue**.

Step 4: Store Message

The message is stored permanently in the **Key-Value Store**.

Step 5: Deliver Message

Case A: User B is Online

- The message is forwarded to **Chat Server 2**
- User B is connected to Chat Server 2 through WebSocket
- Chat Server 2 immediately delivers the message

Case B: User B is Offline

- The system triggers the **Push Notification (PN) Server**
- A push notification is sent to User B's mobile device

Step 6: Receive Message

Chat Server 2 forwards the message to User B through the active WebSocket connection.

Message Synchronization Across Multiple Devices

Modern chat applications support multiple devices for the same account.

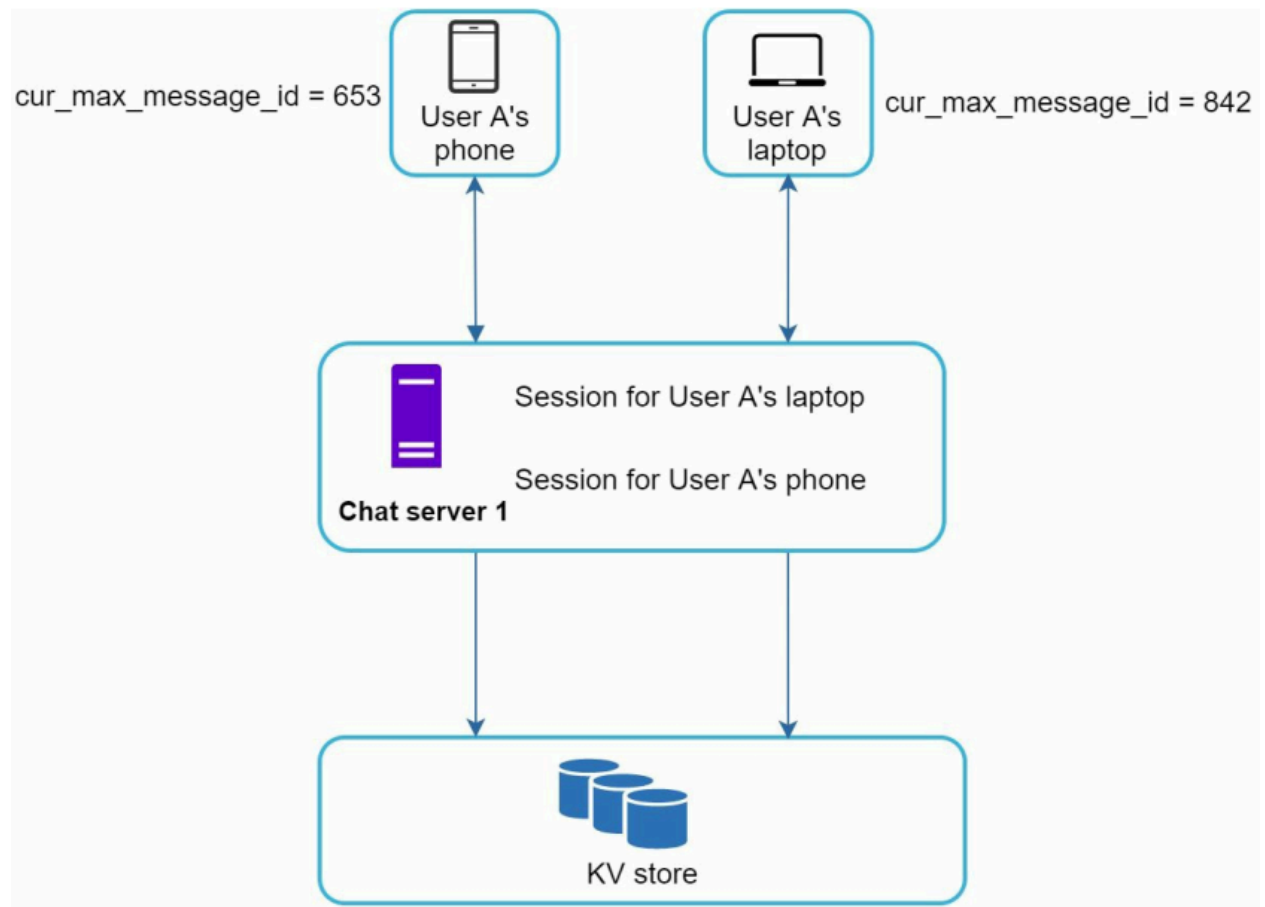


Figure shows synchronization between devices.

Example Scenario

User A is logged in from:

- Phone
- Laptop

Both devices establish separate WebSocket connections with Chat Server 1.

`cur_max_message_id`

Each device maintains a variable called:

None

`cur_max_message_id`

It stores the latest message ID already received on that device.

Conditions for New Messages

A message is considered new if:

1. The recipient ID matches the logged-in user ID
2. The message_id is greater than cur_max_message_id

Synchronization Process

Phone

- Current max message ID = 100

Laptop

- Current max message ID = 95

If new messages with IDs:

None

`96, 97, 98, 99, 100, 101`

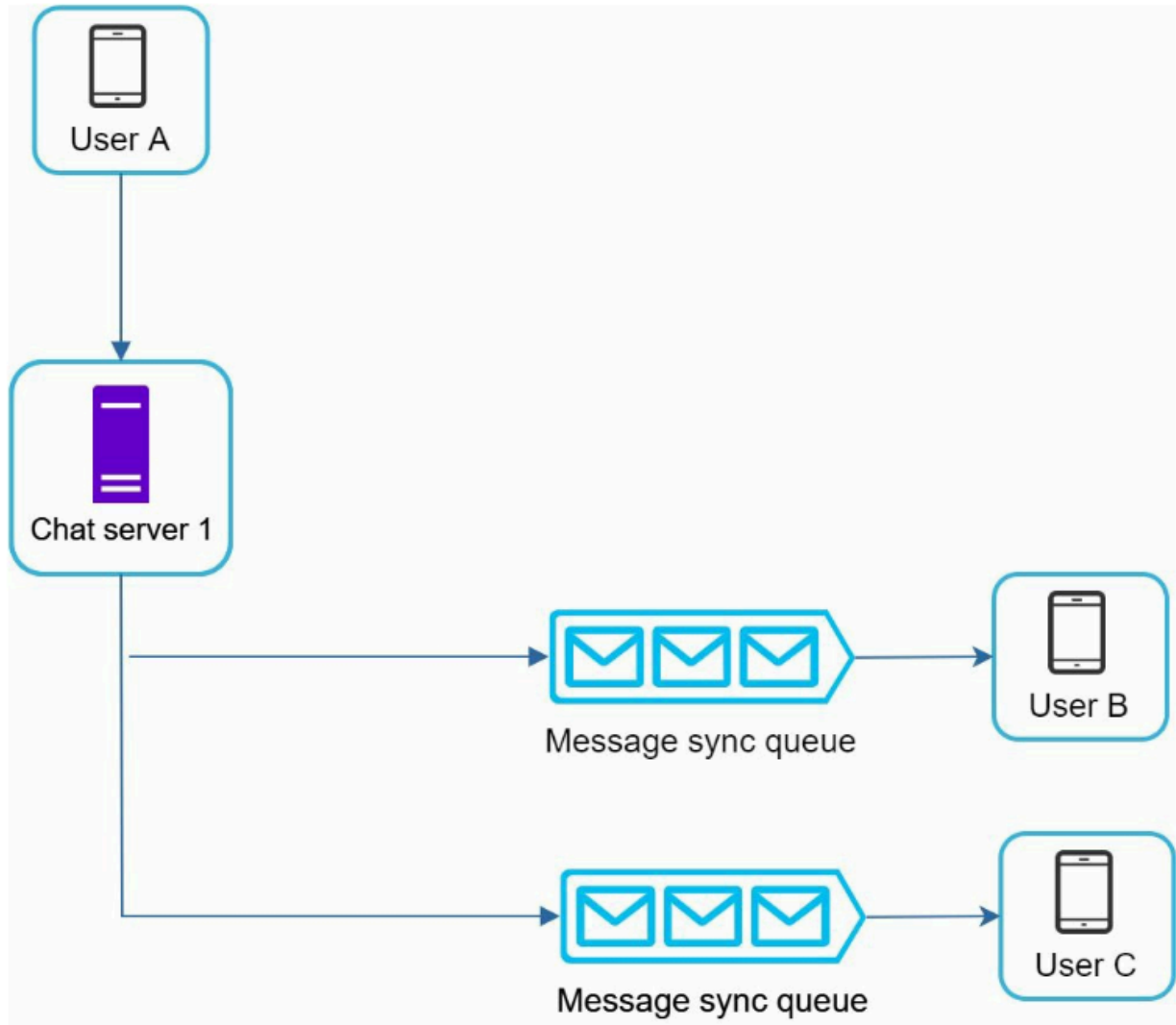
exist in the KV store:

- Laptop fetches messages 96–101
- Phone fetches only message 101

This mechanism ensures efficient synchronization across devices.

Small Group Chat Flow

Group chat is more complex than one-on-one chat.



Group Message Sending Flow

Assume a group contains:

- User A
- User B
- User C

User A sends a message.

Workflow

Step 1: Send Group Message

User A sends the message to the chat server.

Step 2: Copy Message to Recipient Queues

The system creates copies of the message for:

- User B's message sync queue
- User C's message sync queue

Each recipient has a personal inbox queue.

Why This Design Works Well for Small Groups

Advantages

1. Simplified Synchronization

Each user only checks their own inbox queue for new messages.

2. Fast Retrieval

Messages are already prepared for each recipient.

3. Easy Offline Support

Offline users can retrieve missed messages directly from their queue.

Limitation

For very large groups, storing copies for every recipient becomes expensive in terms of:

- Storage
- Network bandwidth
- Processing overhead

Therefore, this approach is suitable mainly for **small groups**.

Example: WeChat uses a similar design and limits groups to around 500 members.

Recipient Side Flow

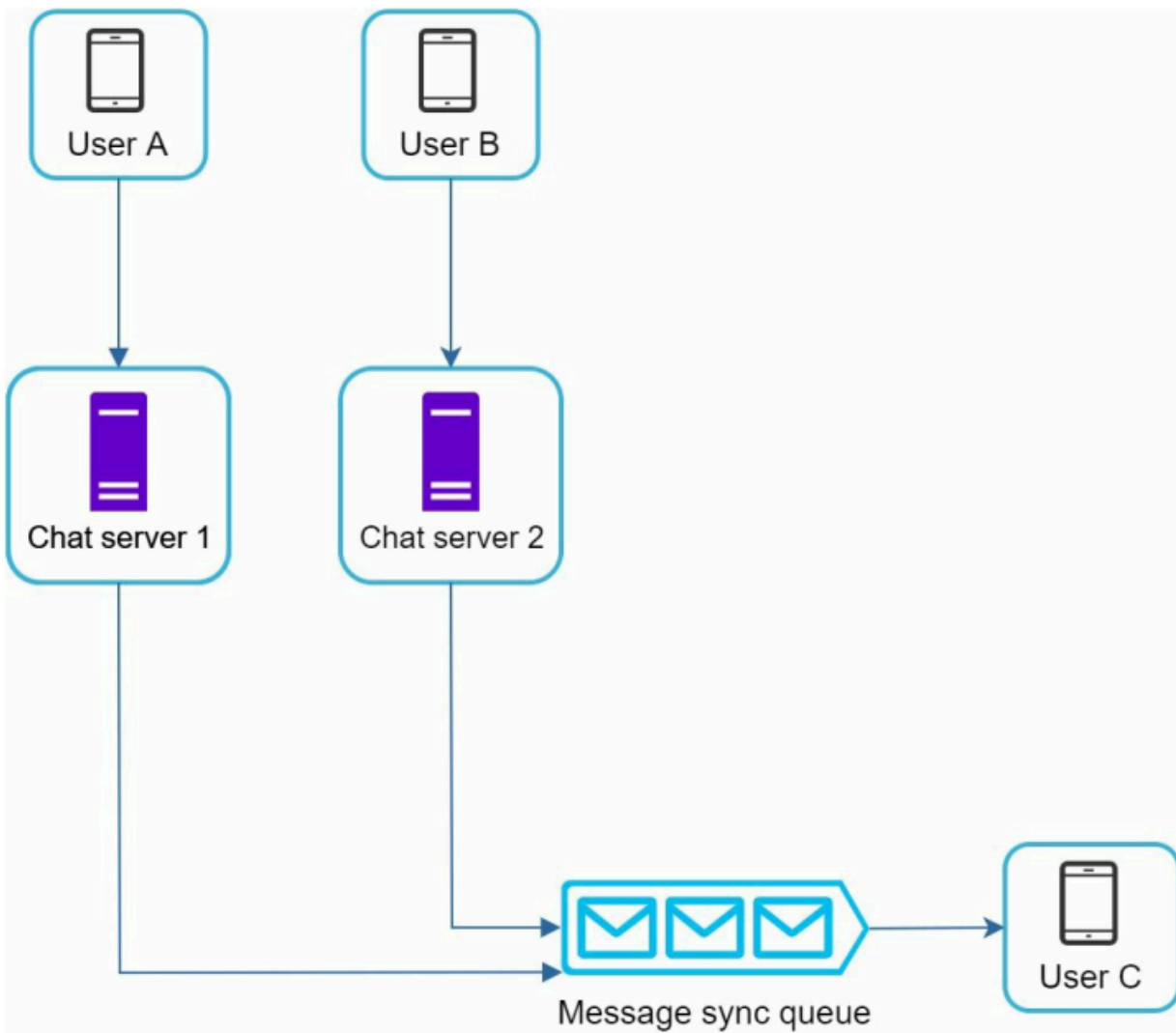


Figure shows the recipient-side architecture.

Each user maintains:

- One inbox/message sync queue
- Messages from multiple senders stored together

Example:

None

User B Inbox:

- Message from User A
- Message from User D
- Message from User F

The client continuously checks this queue to retrieve and display new messages.

Key Advantages of the Overall Design

1. Low Latency

Persistent WebSocket connections enable real-time communication.

2. Reliable Delivery

Messages are stored in a durable Key-Value Store.

3. Multi-Device Support

Each device independently synchronizes messages.

4. Offline Support

Push notifications and sync queues ensure messages are not lost.

5. Scalability

Message queues decouple components and improve scalability.